Hewlett Packard
Enterprise

HPE Cray EX

Programming and Optimization

Lab 4 – Perftools with PBS

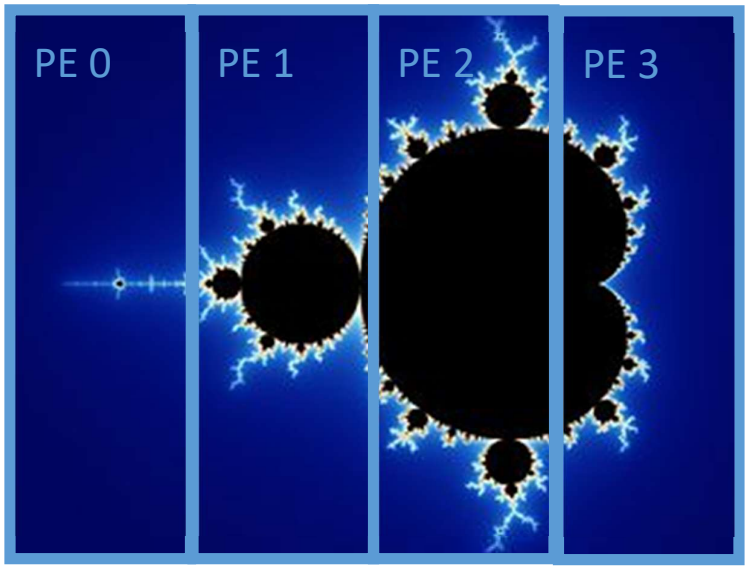
July 2, 2026

Profiling an unbalanced application

In this lab we will be using mandelbrot_mpi.c which checks whether complex numbers are in the Mandelbrot set.

The Mandelbrot set is the set of complex numbers $\mathbb{C}$ *for which the function* $f_c(Z) = Z^2 + C$ *does not diverge to infinity when iterated from* $Z = 0$*, i.e., for which the sequence* $f_c(0), f_c(f_c(0))$*, etc., remains bounded in absolute value.*
https://en.wikipedia.org/wiki/Mandelbrot_set

The algorithm iterates over each number in the complex plane until it determines that a number isn't in the Mandelbrot set or hits a max iteration. The work is distributed across MPI ranks by column. Each MPI Rank processes the same number of columns in the grid but not all columns require the same amount of work



We can clearly see that this approach is unbalanced. We will use perftools to prove it

Exercise 3: Perftools

Step 1 □

Log in to the MPP login node at 192.168.218 20 with your assigned training credentials.
Training user name **trngX** - where X is 1 -15
Training user password **PassW0rdX**

Note: These exercises use the Bourne Again Shell (bash)

Step 2 □

This lab will use the Cray Performance Analysis Tools (perftools) to analyze the performance of a C program `mandelbrot_mpi.c` a copy of this program is included at the end of the lab for your reference.

Create a directory for your work in this lab and copy the `mandelbrot_mpi.c` program to it from a location provided by your instructor. Then verify that PrgEnv-cray is loaded in your environment and that you are targeting the correct hardware. Finally load the `perftools-lite` module.

```
~> mkdir perftools-lab
~> cd pertools-lab
~/pertools-lab> cp /apps/hpe-training/mandelbrot_mpi.c .
~/pertools-lab> module list
~/pertools-lab> module swap craype-x86-rome craype-x86-
spr-hbm
~/pertools-lab> module load perftools-lite
```

Step 3 □

Compile mandelbrot_mpi.c using the cc compiler driver.

```
~> cc -o mandelbrot mandelbrot_mpi.c
```

What is the temporary directory that object files were saved to?_____

Sample pbs batch script run_mb.sh :

```
#!/bin/bash
#PBS -l select=8:ncpus=8
#PBS -j oe
#PBS -q workq
#PBS -l walltime=00:05:00

cd $PBS_O_WORKDIR
module use /opt/cray/pals/modulefiles/
module load cray-pals

echo "----------------------"
mpiexec  -n 8 ./mandelbrot
```

Step 4 □

Run `mandelbrot_mpi.c` MPI processes to generate a perftools lite report. You can use a batch script like the sample provided above.

```
~/pertools-lab> qsub run_mb.sh
~/pertools-lab> less run_mb.sh.JOB_ID
```

	When the job finishes, review the perftools lite report in the job output file and answer the following questions. How many sample hits, on average, were there in the function `MPI_REDUCE`?_____ Which line number within the main routine had the most sample hits? _____

Step 5
□
	Switch modules from `perftools-lite` to `perftools`. Then recompile your program and instrument it with `pat_build` and then run the instrumented version on 16 ranks. If running in a batch script, be sure to update the executable name.
	
	`~/pertools-lab> module swap perftools-lite perftools`
	`~/pertools-lab> module list`
	`~/pertools-lab> cc -o mandelbrot mandelbrot_mpi.c`
	`~/pertools-lab> pat_build mandelbrot`
	`~/pertools-lab> vi run_mb_pbs.sh`
	`~/pertools-lab> qsub run_mb.sh`
	
	What is the name of the directory created by your run? _____
	Note: Its specified in the output file for the job

Step 6 □	Run `pat_report` on your experiment directory to generate a report and more. Be sure to redirect output to save the report that is printed to standard out. `~/pertools-lab> pat_report -i mandelbrot+pat mandelbrot+pat+NUMBERs > mandelbrot-rpt1` Answer the following questions based on the report generated. According to table 2, which rank had the most samples in `MPI_REDUCE`? _____ Which rank had the fewest samples in `MPI_REDUCE`? _____ According to table 4, which hardware performance counter had the most activity in this experiment ? _____
Step 7 □	Look at the automatic profiling analysis file generated by `pat_report` in the experiment directory. Answer the following questions assuming the next tracing experiment will use the settings specified in this file. `~/pertools-lab> cat mandelbrot+pat+NUMBERs/build-options.apa` Which group of performance counters (PERFCTR) will be traced next? _____ Which non-user defined librarie(s) will be traced next? _____ Which user defined library will be traced next? _____

Step 8
□
	Instrument mandelbrot using the automatic profiling analysis file generated by `pat_report` in step 6. When completed run pat_report on your new experiment directory, again being sure to capture the report written to standard out. Answer the questions below based on this latest report and by comparing the latest report to the report from your (initial)sampling run.
	Hint - using `build_options.apa` will change the name of your executable.
	`~/pertools-lab> pat_build -O mandelbrot+pat+NUMBERSs/build-options.apa`
	`~/pertools-lab> vi run_mb_pbs.sh # change executable name`
	
	How much time, on average, was spent in the MPI_REDUCE function? _____
	
	What is the difference between table 1 in the sampling report and table 1 in the tracing report? _____
	
	Which rank spent the least amount of time in the MPI_REDUCE function? _____
	
	Look back at which hardware performance counter had the most activity in your sampling experiment, for that same counter, in which traced function did most of that activity take place? _____

The following steps are advanced activities that you could try if you would like additional practice. They can be run in any order	
Step 9 □	Rerun mandelbrot with custom MPI rank reordering • Review the `MPICH_RANK_REORDER.USER_time` file in the experiment directory created in step 8 • Follow the instructions to update your job script to use the custom reordering o No need to recompile or rebuild your executable for this experiment • Run pat_report on your latest experiment and see if your performance improved
Step 10 □	Rerun mandelbrot to track additional hardware performance counters • run `papi_native_avail` to see which hardware performance counters are available to collect on the compute nodes • Review job output and select a new counter or two that interest you • update your job script to change the environment variable `PAT_RT_PERFCTR` to reference your selected variables • e.g. "`export PAT_RT_PERFCTR=PAPI_VAR1,PAPI_VAR2,...`" • Rerun your experiment and generate a report to see your selected hardware performance counters
Step 11 □	Visualize experiments with Apprentice 2 • Be sure you have x11 forwarding enabled • Select an experiment directory from one of your experiments and run App2: •

```
% ssh -X training1XX@UAN
% app2 mandelbrot+apa+97110-3921t/
```

- Review the available visualizations and be sure to hover over parts of each for additional details
- Try comparing this experiment to another of your experiments with the compare option

mandelbrot_mpi.c source code:

```
/* Program to calculate the Mandlebrot set with MPI parallelism
   D. Acreman */

#include <stdio.h>
#include <complex.h>
#include <stdbool.h>
#include <math.h>
#include <mpi.h>

/* Number of points on real and imaginary axes*/
#define N_RE 12000
#define N_IM  8000

/* Number of iterations at each z value */
int nIter[N_RE+1][N_IM+1];

/* Buffers for MPI communication */
int sendBuffer[(N_RE+1)*(N_IM+1)];
int receiveBuffer[(N_RE+1)*(N_IM+1)];

/* Domain size */
const float z_Re_min = -2.0; /* Minimum real value*/
const float z_Re_max =  1.0; /* Maximum real value */
const float z_Im_min = -1.0; /* Minimum imaginary value */
const float z_Im_max =  1.0; /* Maximum imaginary value */

/* Logical variable to control program behaviour */
const bool doIO    = false; /* Set to true to write out results */
const bool verbose = false; /* Set to true for more verbose output */

/*****************************************************************************/

int main(int argc, char *argv[]){

  /* Loop indices for real and imaginary axes*/
  int i, j;
  /* Loop index for iteration*/
  int k;
  /* Maximum number of iterations*/
  const int maxIter=100;

  /* Real and imaginary components of z*/
  float z_Re, z_Im;
  /* Value of Z at current iteration*/
  float complex z;
  /*Value of z at iteration zero*/
  float complex z0;

  /* Rank of this MPI process */
  int myRank;
  /* Total number of MPI processes*/
  int nProcs;
```

```
  MPI_Init(&argc, &argv);

  /* Record start time */
  double start_time, end_time;
  start_time=MPI_Wtime();

  MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
  MPI_Comm_size(MPI_COMM_WORLD, &nProcs);

  if (verbose) {printf("Process %d of %d starting calculation\n", myRank, nProcs);}

  /* Initialise to nIter zero as we will use a reduce to collate results into this
array*/
  for (i=0; i<N_RE+1; i++){
    for (j=0; j<N_IM+1; j++){
      nIter[i][j]=0;
    }
  }

  /* Divide work between MPI processes by splitting the i loop into nProcs pieces */
  int nRePerProc = N_RE / nProcs;
  int imin = nRePerProc * myRank;
  int imax = nRePerProc * (myRank+1) - 1;
  if ( myRank==nProcs-1 ) imax=N_RE;

  if (verbose) {printf("Process %d will calculate %d to %d\n", myRank, imin, imax);}

  /* Loop over real and imaginary axes */
  for (i=imin; i<imax+1; i++){
    for (j=0; j<N_IM+1; j++){
      z_Re = ( ( (float) i)/ ( (float) N_RE)) * (z_Re_max - z_Re_min) + z_Re_min;
      z_Im = ( ( (float) j)/ ( (float) N_IM)) * (z_Im_max - z_Im_min) + z_Im_min;
      z0 = z_Re + z_Im*I;
      z  = z0;

      /* Iterate up to a maximum number or bail out if mod(z) > 2 */
      k=0;
      while(k<maxIter){
        nIter[i][j] = k;
        if (cabs(z) > 2.0)
          break;
        z = z*z + z0;
        k++;
      }
    }
  }

  /* call MPI_reduce to collate all results on process 0*/
  int index=0;
  /* Pack nIter into a 1D buffer for sending*/
  for (i=0; i<N_RE+1; i++){
    for (j=0; j<N_IM+1; j++){
      sendBuffer[index]=nIter[i][j];
      index++;
    }
  }

  MPI_Reduce(&sendBuffer, &receiveBuffer, (N_RE+1)*(N_IM+1), MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

  /* Unpack receive buffer into nIter */
  index=0;
  for (i=0; i<N_RE+1; i++){
    for (j=0; j<N_IM+1; j++){
      nIter[i][j]=receiveBuffer[index];
      index++;
```

```
    }
  }

  /* Write out results */
  if (doIO && myRank==0 ){
    if (verbose) {printf("Writing out results from process %d \n", myRank);}
    FILE *outfile;
    outfile=fopen("mandelbrot.dat","w");

    for (i=0; i<N_RE+1; i++){
      for (j=0; j<N_IM+1; j++){
        z_Re = ( ( (float) i)/ ( (float) N_RE)) * (z_Re_max - z_Re_min) + z_Re_min;
        z_Im = ( ( (float) j)/ ( (float) N_IM)) * (z_Im_max - z_Im_min) + z_Im_min;
        fprintf(outfile,"%f %f %d \n",z_Re, z_Im, nIter[i][j]);
      }
    }
    fclose(outfile);
  } else {
    if (myRank==0){
      printf("Centre: nIter=%d\n",nIter[N_RE/2][N_IM/2]);
        }
  }

  /* Record end time. The barrier synchronises the process so they all measure the same
time */
  MPI_Barrier(MPI_COMM_WORLD);
  end_time=MPI_Wtime();

  /* Report elapsed time */
  if (myRank==0){
    printf("STATS (num procs, elapsed time): %d %f\n", nProcs, end_time-start_time);
  }

  MPI_Finalize();

  return 0;

}
```